

Senior DevOps/Backend SWE (Contract)

Bespoke Labs | Take Home Test

Submission Link: <https://forms.gle/Jfsy546UKWJb9276>

To Note 1: Please ensure that you don't plagiarise. We have strict plagiarism checkers which will blacklist you if they indicate cheating. We check for existing tasks i.e. from existing submissions and harbor/terminal bench 1.0 or 2.0 task registry as well.

To Note 2: This assessment will be similar to what you work on later in the project.

To Note 3: If you're able to submit this test within 2 days of receipt, pass the requirements and our internal graders, and are able to work with us for 2 weeks, we will offer you a \$100 joining bonus.

To Note 4: This assessment document is confidential and can not be shared on any public forums. We reserve the right to seek legal help if this is violated.

To Note 5: If you have any questions, please reach out at projects@bespokelabs.ai with the subject "Take Home Test Doubts". Try to make your doubts non-trivial.

1. Objective

Design and implement a "Hard" DevOps or SWE Task based on the Terminal Bench 2.0 Framework. You will create a scenario where an AI agent interacts with a Linux terminal to solve a task in the DevOps or SWE domain.

We define what a "Hard" task entails later in the document.

2. Environment Setup

You must set up the [harbor](#) CLI tool locally to develop and verify your task. Ensure you have docker installed and running.

A. Install UV & Harbor

Shell

```
# 1. Install UV (Python package manager)
curl -LsSf https://astral.sh/uv/install.sh | sh

# 2. Create Environment
uv init
uv venv --python 3.12
source .venv/bin/activate

# 3. Install Harbor CLI
uv tool install harbor

# 4. Verify
harbor --version
```

B. LLM API Setup

- Make an account on console.groq.com.
- Create an API key and log it somewhere.
- *Note: You will be using the LLM `moonshotai/kimi-k2-instruct-0905` for this task.*

Shell

```
# 1. Install the Groq Python Library
uv pip install groq

touch .env
cat GROQ_API_KEY="your_groq_api_key" > .env #to log your API key
EXPORT GROQ_API_KEY="your_groq_api_key"
```

3. Task Creation Instructions

You are required to build a task in the DevOps or the SWE domain. Think of a hard problem statement in either of those domains.

Step 1: Initialize Run the following to generate the skeleton:

Shell

```
harbor tasks init "some_task_name"
```

```
# example usage: harbor tasks init "abc_xyz"
```

This will create a directory with the name abc@xyz.com. Go through it.

Step 2: Go through example tasks [here](#) and get an understanding of what a potential task could look like.

Look at Step 3 for more details on the individual components of a task.

Step 3: Implement Required Files

You must implement the following structure. Do not hardcode solutions inside the environment directory.

- **task.toml**: Task configuration.
 - *Requirement*: Add `[environment.runtime] python = "3.12"`
- **instruction.md**: The prompt given to the AI.
 - *Tip*: Be descriptive about the *goal* (e.g., "Fix the build"), but do not reveal the *steps* (e.g., "Downgrade numpy to v1.21").
 - Refer to any files in the environment/ folder as `/app/file_name` (since you will be copying the environment into `/app/` in the Docker container).
 - If your solution needs to create any files, they should also be in `/app/` directory.
- **environment/**: The Docker environment.
 - This is what the AI Agent has access to beyond the instruction.md.
 - Think of this as the base environment/set of files/code that the agent will try to work on top of. This can be a microservice, a set of microservices, a web-application, etc.
 - Use the **Dockerfile** to setup the environment with all package installations, environment build commands and ensure you copy all relevant files into `/app/`, ie. add ``COPY * /app/`` to the Dockerfile.
 - *Constraint*: Do not include the solution or test files in the image build or in the *environment* folder.

- **solution/**: Golden Solution (This is the ideal human written solution).
 - Create a script/set of scripts that solves the problem defined in *instruction.md*.
 - Ensure you invoke these scripts in *solve.sh*.
 - This is just to check if your environment builds, and your solution is able to pass tests.
- **tests/**: Verification Logic.
 - This will include unit tests(in pytest format) that will verify the solution – yours(*solve.sh*) and the AI Agent that attempts the problem defined in [instruction.md](#).
 - If your solution required creating any files as output, they should be referenced with the path `/app/{solution_output_file_name}` in the *test_outputs.py* file.
 - Do not touch the *test.sh* file.
 - Add all your tests into *test_outputs.py*.

3. Run Task

Step 1: Run the Oracle (It just runs your *solve.sh* and verifies it against the *tests/*. This doesn't use any AI Agent).

Run this outside the task directory.

Shell

```
harbor run -p ". /abx@xyz.com" -a oracle
```

This should pass. If it gives an error log, there is an issue in your task creation step. Go back and try to debug. Look at <https://harborframework.com/docs/>.

If it passes, move to Step 2.

Step 2: Run the task but this time the AI Agent will try to solve for it.

Run this outside the task directory.

Shell

```
harbor run -p "./abc@xyz.com" -a terminus-2 --model  
groq/moonshotai/kimi-k2-instruct-0905 -k 10 -n 10
```

Ensure it is giving you a report at the end of it. k=10 allows you to run it 10 times, you can test it with a lesser number of attempts as well initially. k=10 is important to meet difficulty requirements specified below. `-n` is concurrent threads for the k attempts.

If you run out of API credits on Groq, please add some credits and you can get it reimbursed if you qualify the difficulty guidelines mentioned below.

4. Validation (Sanity Checks)

Difficulty Guidelines:

- The task must be solvable but difficult.
- Target Score: On 10 different runs with a standard LLM agent, the average success rate should be > 0.0 but < 0.7 .
- The agent must demonstrate reasoning (e.g., analyzing logs, resolving circular dependencies) rather than simple file manipulation.

Quality Checklist [**Please use claude code to automate this QC for you if feasible**]

1. **[Important]** Test suite should cover everything asked in **instruction.md**
2. **[Important]** test suite should do **functional tests**
3. **[Important]** Make sure to add multiple functional tests
4. Reproducibility: Use pinned versions in your Dockerfile (e.g., `pip install numpy==1.21.0`, not `latest`).
5. No Cheating: Ensure the agent cannot find the answer by purely `cat`-ing a file in the environment/ directory. The agent will have access to the environment/ directory and the instruction.md.
6. Isolation: Ensure `tests/` folder is not copied into the Docker image; the agent should not see the test logic.
7. Error Handling: If your task involves failure scenarios around "Disk Space" or "Permissions," ensure the environment actually reproduces these errors, rather than simulating them.

5. Submission Workflow

Submit here: <https://forms.gle/Jfsy546UKWJb9276>

1. Prepare Folder: Ensure your task folder contains `task.toml`, `instruction.md`, `solution/`, `tests/`, and `environment/`.
2. Zip: Compress the entire task folder into a single `.zip` file.
3. Submit: Upload the ZIP into the submission form provided in this document along with your details.